

Lab 8 - Uso de LLMs e Prompt Engineering

Parte 1 - Informações sobre a ferramenta

Nome da ferramenta: GPT-5.5 Instant

Data de uso: 13/06/2026

Versão: Gratuita

Acesso à internet: Não

Parte 2 - Primeira tarefa: respostas conceituais

Prompt simples

"Quem é Alan Turing?"

Resposta do GPT-5.5 Instant

Alan Turing foi um matemático, lógico, criptanalista e pioneiro da computação. Ele é frequentemente considerado uma das pessoas mais importantes da história da ciência da computação.

Principais contribuições

1. Fundamentos da computação

Em 1936, Turing descreveu a chamada **Máquina de Turing**, um modelo teórico capaz de executar qualquer algoritmo que possa ser computado. Essa ideia se tornou a base conceitual dos computadores modernos.

2. Quebra de códigos na Segunda Guerra Mundial

Durante a Segunda Guerra Mundial, Turing trabalhou em Bletchley Park, onde ajudou a decifrar mensagens criptografadas pela máquina alemã Enigma. Muitos historiadores acreditam que esse trabalho encurtou a guerra em vários anos e salvou milhões de vidas.

3. Inteligência artificial

Em 1950, Turing publicou o artigo *Computing Machinery and Intelligence*, propondo o famoso **Teste de Turing**, um experimento para avaliar se uma máquina consegue exibir comportamento inteligente indistinguível do de um ser humano.

Vida pessoal e perseguição

Turing era homossexual em uma época em que a homossexualidade era crime no Reino Unido. Em 1952, ele foi processado por sua orientação sexual e submetido a um tratamento hormonal compulsório. Sua carreira foi prejudicada e ele perdeu acesso a trabalhos sigilosos do governo.

Em 1954, Turing morreu aos 41 anos. A causa oficial foi envenenamento por cianeto, geralmente considerada suicídio, embora existam debates sobre as circunstâncias exatas.

Reconhecimento posterior

Décadas depois, o governo britânico reconheceu a injustiça cometida contra ele:

- Em 2009, o então primeiro-ministro Gordon Brown apresentou um pedido oficial de desculpas.
- Em 2013, Turing recebeu um perdão real póstumo.
- Em 2021, sua imagem passou a estampar a nota de £50 do Bank of England.

Por que ele é tão importante?

Turing teve impacto direto em três áreas gigantes:

1. **Computação** - ajudou a definir o que significa computar.
2. **Criptografia** - contribuiu decisivamente para a vitória dos Aliados na guerra.
3. **Inteligência Artificial** - lançou algumas das primeiras perguntas sérias sobre máquinas pensantes.

Por isso, ele é frequentemente chamado de "**pai da ciência da computação**".

Ferramenta utilizada: GPT-5.5 Instant

Data: 13/06/2026

Trecho da resposta aproveitado: A resposta inteira, exceto a introdução da resposta (Ah, claro que posso ajudar!) e a pergunta que a IA fez ao final ("Precisa de mais alguma ajuda?")

Como a resposta foi verificada: Pesquisas na internet para cruzar as principais informações apresentadas

Prompt elaborado

"Me faça uma explicação detalhada sobre quem foi Alan Turing e a sua imensa contribuição para a sociedade atual. Gostaria que focasse um pouco também na perseguição que o mesmo sofreu por conta de ser homossexual. O trabalho é para uma apresentação na disciplina de "Introdução à Ciência da Computação" para os meus colegas (Sou um aluno)."

Resposta do GPT-5.5 Instant

Para uma apresentação de **Introdução à Ciência da Computação**, vale a pena mostrar que Alan Turing não foi apenas um matemático brilhante: ele ajudou a criar os fundamentos da computação moderna e teve uma vida marcada por uma grave injustiça devido à sua orientação sexual.

Quem foi Alan Turing?

Alan Turing nasceu em Londres, na Inglaterra, em 23 de junho de 1912. Foi matemático, lógico, criptógrafo e cientista da computação.

Embora tenha vivido apenas 41 anos, suas ideias moldaram praticamente toda a tecnologia digital que utilizamos atualmente: computadores, smartphones, internet, inteligência artificial e muito mais.

Hoje ele é frequentemente chamado de **"pai da Ciência da Computação"**.

A Máquina de Turing: o nascimento da computação teórica

Em 1936, Turing publicou um artigo revolucionário chamado *On Computable Numbers*.

Nesse trabalho, ele imaginou um dispositivo teórico conhecido como **Máquina de Turing**.

A ideia era simples, mas extremamente poderosa: uma máquina capaz de ler instruções e manipular símbolos seguindo regras bem definidas.

O mais impressionante é que Turing demonstrou que uma única máquina poderia simular qualquer outra máquina computacional, desde que recebesse as instruções corretas.

Esse conceito é a base dos computadores modernos.

Em outras palavras:

- Antes de Turing, não existia uma definição clara do que significava "computar".
- Turing criou um modelo matemático para descrever qualquer processo computacional.
- Os computadores atuais seguem exatamente o mesmo princípio: hardware genérico executando programas diferentes.

Por isso, quando você abre um navegador, um jogo ou um editor de texto, está utilizando uma ideia que remonta diretamente ao trabalho de Turing.

O papel de Turing na Segunda Guerra Mundial

Durante a Segunda Guerra Mundial, Turing trabalhou em Bletchley Park, centro secreto britânico de criptografia.

Os alemães utilizavam uma máquina chamada Enigma para codificar mensagens militares.

Acreditava-se que suas mensagens eram praticamente impossíveis de decifrar.

Turing liderou o desenvolvimento de máquinas eletromecânicas chamadas "Bombes", capazes de acelerar enormemente o processo de quebra desses códigos.

O resultado foi extraordinário:

- Os Aliados passaram a ler diversas comunicações alemãs.
- Isso permitiu antecipar ataques e movimentações militares.
- Muitos historiadores estimam que o trabalho realizado em Bletchley Park encurtou a guerra em cerca de dois anos.
- Milhões de vidas podem ter sido salvas como consequência.

Durante décadas, esse trabalho permaneceu em segredo devido ao sigilo militar.

A contribuição para a Inteligência Artificial

Turing também foi um dos primeiros cientistas a perguntar:

"As máquinas podem pensar?"

Em 1950, publicou o artigo *Computing Machinery and Intelligence*, que introduziu o famoso **Teste de Turing**.

Teste de Turing consiste em uma situação em que uma pessoa conversa com um humano e uma máquina sem saber quem é quem.

Se a máquina conseguir convencer o avaliador de que é humana, ela demonstraria um comportamento inteligente.

Mesmo décadas depois, o Teste de Turing continua sendo uma das referências mais conhecidas nas discussões sobre inteligência artificial.

Como suas ideias influenciam a tecnologia atual

Praticamente toda a computação moderna está ligada aos conceitos desenvolvidos por Turing.

Alguns exemplos:

Computadores pessoais

Os computadores utilizam programas armazenados e executados por uma máquina universal, exatamente como Turing imaginou.

Smartphones

Cada aplicativo é apenas um conjunto diferente de instruções executadas pelo mesmo dispositivo.

Internet

Servidores, protocolos e sistemas digitais dependem dos princípios da computação formal desenvolvidos no século XX.

Inteligência Artificial

Grande parte da pesquisa em IA surgiu a partir das perguntas levantadas por Turing sobre inteligência e máquinas.

Segurança Digital

A criptografia moderna evoluiu diretamente da área em que Turing trabalhou durante a guerra.

A perseguição por ser homossexual

Apesar de suas contribuições extraordinárias, Turing sofreu uma enorme injustiça.

Na Inglaterra dos anos 1950, atos homossexuais entre homens eram considerados crime.

Em 1952, Turing foi denunciado às autoridades após revelar que mantinha um relacionamento com outro homem.

Ele foi acusado de "indécência grave", uma infração criminal da época.

Como alternativa à prisão, o governo ofereceu um tratamento hormonal compulsório, conhecido popularmente como **castração química**.

As consequências foram severas:

- Alterações físicas causadas pelos hormônios.
- Impactos psicológicos significativos.
- Perda de autorizações de segurança do governo.
- Humilhação pública.
- Interrupção de parte de sua carreira científica.

É importante destacar que o mesmo Estado que havia se beneficiado enormemente de seu trabalho durante a guerra passou a persegui-lo por sua orientação sexual.

Morte e reconhecimento tardio

Em 1954, aos 41 anos, Turing foi encontrado morto em sua casa.

A investigação da época concluiu que sua morte ocorreu por envenenamento por cianeto. Ainda existem debates históricos sobre as circunstâncias exatas, mas a conclusão oficial foi de suicídio.

Décadas depois, o Reino Unido começou a reconhecer a injustiça que havia cometido.

Marcos importantes:

- Em 2009, o governo britânico apresentou um pedido formal de desculpas.
- Em 2013, Turing recebeu um perdão real póstumo.
- Em 2021, sua imagem passou a estampar a nota de 50 libras do Bank of England.

Conclusão

Alan Turing foi um dos cientistas mais importantes da história. Seus trabalhos estabeleceram os fundamentos teóricos da computação, ajudaram os Aliados a vencer a Segunda Guerra Mundial e influenciaram diretamente o desenvolvimento da inteligência artificial.

Ao mesmo tempo, sua história também lembra que grandes avanços científicos podem coexistir com graves injustiças sociais. Embora tenha contribuído enormemente para a humanidade, Turing foi perseguido pelo próprio governo de seu país por ser homossexual.

Hoje, ele é lembrado não apenas como o pai da Ciência da Computação, mas também como um símbolo da luta contra a discriminação e da importância de reconhecer o valor das pessoas independentemente de sua orientação sexual.

Frase final para a apresentação

"Sem Alan Turing, o mundo digital que conhecemos hoje provavelmente não existiria da forma como existe. Sua genialidade transformou a computação, e sua história nos lembra da importância da ciência, da diversidade e dos direitos humanos."

Ferramenta utilizada: GPT-5.5 Instant

Data: 13/06/2026

Trecho da resposta aproveitado: A resposta inteira, exceto a introdução da resposta (Ah, claro que posso ajudar!) e a pergunta que a IA fez ao final ("Precisa de mais alguma ajuda?")

Como a resposta foi verificada: Pesquisas na internet para cruzar as principais informações apresentadas

Análise das respostas

1. A resposta da primeira IA aprofundou pouco todos os tópicos, já que não solicitei um formato de por exemplo "ter que ser didático para uma apresentação" e nem mesmo solicitei os pontos principais que eu gostaria de destacar.
2. A resposta da segunda IA conseguiu aprofundar melhor que a primeira em quase todos tópicos, já que eu especifiquei uma situação para ele gerar os dados (como objetivo do texto, qual curso eu faço, apresentação para quem, etc.), além de que ele acrescentou muito mais informações no assunto que eu dei ênfase. Portanto, a segunda resposta ficou mais completa, mais didática, mais profissional e também possui assuntos de que eu (apresentador do trabalho) tenho interesse de enfatizar.

Parte 3 - Segunda tarefa: geração de código

Prompt utilizado

"Faça um código-fonte de um programa que diz qual é o valor do termo n da sequência de Fibonacci"

Código gerado

[Resposta do GPT-5.5 Instant]


```
#include <stdio.h>

int main() {
    int n;
    unsigned long long a = 0, b = 1, temp;

    printf("Digite o valor de n: ");
    scanf("%d", &n);

    if (n < 0) {
        printf("n deve ser maior ou igual a 0.\n");
        return 1;
    }

    if (n == 0) {
        printf("Fibonacci(%d) = 0\n", n);
        return 0;
    }

    if (n == 1) {
        printf("Fibonacci(%d) = 1\n", n);
        return 0;
    }

    for (int i = 2; i <= n; i++) {
        temp = a + b;
        a = b;
        b = temp;
    }

    printf("Fibonacci(%d) = %llu\n", n, b);

    return 0;
}
```

[Fim da resposta]

Ferramenta utilizada: GPT-5.5 Instant

Data: 13/06/2026

Trecho da resposta aproveitado: Apenas o código fornecido

Como a resposta foi verificada: Análise do código com base em conhecimentos prévios e teste manual com exemplos de inputs

Explicação do código:

O programa pede pro usuário digitar um valor n e calcula o n -ésimo termo da sequência de Fibonacci usando um laço for. Os dois primeiros termos ($F(0) = 0$ e $F(1) = 1$) são tratados separadamente com ifs, e pra todo o resto ele vai somando os dois últimos valores até chegar no termo que a gente quer.

1º Teste:

Digite o valor de n : 10

Fibonacci(10) = 55

2º Teste:

Digite o valor de n : 0

Fibonacci(0) = 0

Ambos os testes deram resultado correto.

Análise crítica da solução

O código funciona pra valores normais, mas tem alguns problemas que me chamaram a atenção:

1. O código foi gerado em C. O Chat-GPT não perguntou ao usuário qual seria a linguagem desejada antes de realizar a execução;
2. Se o usuario colocar alguma letra o programa vai dar erro/retornar algum lixo de memória;
3. Long long é completamente ineficiente para esse tipo de exercício, podendo ocasionar estouro de memoria dependendo da entrada mais facilmente;
4. O código não é comentado em nenhum momento.

Parte 4 - Melhorando o prompt

Prompt modificado

"Escreva um programa em Python que calcula o valor do termo n da sequência de Fibonacci. Inclua comentários no código explicando cada bloco lógico. Mostre exemplos de entrada e saída. O programa deve tratar erros como entrada negativa, não-inteira ou muito grande. Não use bibliotecas externas, use um laço while e não recursão."

Elementos adicionados em relação ao prompt anterior

Adicionei cinco coisas que faltavam no prompt original:

1. Linguagem de programação: especificar Python em vez de deixar a IA escolher sozinha.
2. Comentários: pedir explicação de cada bloco lógico.
3. Exemplos de entrada e saída: pra demonstrar como o programa funciona.
4. Tratamento de erros: entrar negativo, texto, número muito grande.
5. Restrições de implementação: usar while em vez de for, sem recursão, sem bibliotecas externas.

Código gerado

```
# Programa para calcular o termo n da sequência de Fibonacci

# Define um limite para evitar cálculos exageradamente grandes
LIMITE_MAXIMO = 100000

# Solicita a entrada do usuário
entrada = input("Digite o valor de n: ")

try:
    # Tenta converter a entrada para inteiro
    n = int(entrada)

    # Verifica se o número é negativo
    if n < 0:
        print("Erro: n deve ser um número inteiro não negativo.")

    # Verifica se o número é muito grande
    elif n > LIMITE_MAXIMO:
        print(f"Erro: n deve ser menor ou igual a {LIMITE_MAXIMO}.")

    else:
        # Casos base da sequência
        if n == 0:
            fibonacci = 0
        elif n == 1:
            fibonacci = 1
        else:
            # Inicializa os dois termos anteriores
            anterior = 0
            atual = 1

            # Contador para percorrer a sequência
            contador = 2

            # Calcula iterativamente até o termo n
            while contador <= n:
                proximo = anterior + atual
                anterior = atual
                atual = proximo
                contador += 1
```

```
fibonacci = atual

# Exibe o resultado
print(f"0 termo F({n}) = {fibonacci}")

except ValueError:
    # Executado caso a entrada não seja um número inteiro válido
    print("Erro: digite um número inteiro válido.")
```

Ferramenta utilizada: GPT-5.5 Instant

Data: 13/06/2026

Trecho da resposta aproveitado: Quase a resposta inteira, exceto pelo parágrafo introdutório, mas principalmente o código e os exemplos

Como a resposta foi verificada: Análise do código com base em conhecimentos prévios e teste manual

Explicação do código

O código segue uma lógica bem direta:

1. Lê a entrada do usuário, depois faz uma série de validações (converte pra inteiro, rejeita negativos e números muito grandes).
2. trata os casos base ($F(0) = 0$ e $F(1) = 1$), e pra todo o resto usa um laço while pra ir somando os dois últimos termos até chegar no n-ésimo.
3. No final, exibe o resultado. Se a entrada não for um número inteiro, cai no except ValueError e manda a mensagem de erro.

Testes realizados

1º Teste:

Digite o valor de n: 10

O termo $F(10) = 55$

2º Teste:

Digite o valor de n: 0

O termo $F(0) = 0$

3º Teste:

Digite o valor de n: -5

Erro: n deve ser um número inteiro não negativo.

4º Teste:

Digite o valor de n: abc

Erro: digite um número inteiro válido.

5º Teste:

Digite o valor de n: 100001

Erro: n deve ser menor ou igual a 100000.

Todos os testes deram o resultado esperado, incluindo os casos de erro.

Comparação entre as duas respostas

A resposta ao prompt elaborado foi mais completa.

1. Dessa vez eu especifiquei Python, então a IA não foi lá e escolheu C do nada.
2. O código tem comentários em cada bloco, coisa que o código passado não tinha nenhum.
3. O tratamento de erros ficou muito mais estrito: no primeiro código, se você digitasse "abc" o programa simplesmente quebrava, enquanto agora ele manda uma mensagem de erro clara.
4. Tem um limite máximo (`LIMITE_MAXIMO = 100000`) pra evitar overflow, o que no código em C simplesmente não existia.
5. O prompt elaborado obrigou a IA a usar `while` em vez de `for`, e a não usar recursão nem bibliotecas externas.
6. No geral, especificar bem o que eu queria fez a resposta ficar consideravelmente melhor. Quanto mais detalhado o prompt, menos margem pra IA inventar coisa ou tomar decisões que não são as ideais pro contexto.

Parte 5 - Verificação crítica

A resposta utilizada pra verificação crítica foi a do prompt simples da Parte 3 (o código em C), porque ele tem problemas maiores pra analisar.

1. **A resposta estava correta?** Sim. A resposta está correta, porém não completa. A lógica do cálculo de Fibonacci tá certa e o programa funciona pra entradas numéricas inteiras positivas. O problema é que o tratamento de erros é insuficiente e ele não se comporta direito pra entradas inválidas como texto ou caracteres especiais.

2. **Havia informações incompletas ou incorretas?** Sim. O código não trata entradas não-numéricas - se a pessoa digitar "abc", o scanf falha e o programa exibe lixo de memória. Também não tem proteção contra overflow: pra n maior que 93, o tipo unsigned long long estoura e gera valores incorretos sem nenhum aviso. E a IA não mencionou nenhuma dessas limitações, apresentou o código como se tivesse pronto.
3. **Como a resposta foi verificada?** Executei o programa com entradas válidas (n = 10 e n = 0) e confirmei que as saídas estavam corretas. Depois testei com valores grandes (n = 94, n = 100) e vi que os resultados estouravam o limite do tipo, gerando valores errados. Também testei com entrada não-numérica ("abc") e o programa não tratou o erro, ficou exibindo lixo de memória. Daí comparei com a resposta do prompt elaborado, que tratou todos esses casos direitinho.
4. **A LLM demonstrou excesso de confiança?** Sim. O GPT-5.5 Instant apresentou o código como uma solução completa, sem alertar sobre overflow nem sobre o fato de que o tratamento de erros era incompleto. A frase "um programa simples em C pode ser:" dá a impressão de que a solução é direta e suficiente, quando na verdade tem falhas que passam batidas se a pessoa não testar direito.
5. **Como o prompt poderia ser melhorado?** Ao especificar a linguagem, pedir tratamento de erros, pedir comentários e colocar restrições de implementação, a resposta ficou bem mais completa.

Parte 6 - Uso responsável de LLMs

1. **Por que não devemos copiar respostas sem revisão?** Porque LLMs podem gerar respostas erradas ou incompletas sem avisar. Como a gente viu na Parte 5, o código em C parecia funcionar, mas tinha falhas de tratamento de erro e overflow que só apareceram quando foram testadas manualmente. Copiar sem revisar é basicamente aceitar como verdade algo que pode estar errado, e isso é perigoso tanto academicamente quando no sentido didático/de aprendizado.
2. **Por que é importante informar os prompts utilizados?** Porque os prompts explicam o contexto em que a IA gerou aquela resposta. Sem saber qual pergunta foi feita, fica difícil avaliar se a resposta é boa ou não. Além disso, informar os prompts é uma questão de ética acadêmica - se eu uso IA pra fazer um trabalho, as outras pessoas precisam saber como essa IA foi utilizada e como tal resposta surgiu à partir da mesma.
3. **Em quais situações as LLMs ajudam no aprendizado?** Pra explicar conceitos abstratos de forma mais didática, pra gerar exemplos de código e testar, pra revisar textos mostrando possíveis erros, para comparar abordagens diferentes de um mesmo problema e também a IA possui "paciência infinita", se você não entendeu algo dá para perguntar quantas vezes quiser, solicitando uma nova abordagem a cada falha no aprendizado.

4. **Em quais situações elas podem atrapalhar?** Quando o aluno copia a resposta sem entender nada, quando a IA gera código com erros sutis que passam despercebidos, quando a pessoa deixa de pensar por conta própria e terceiriza o raciocínio pra máquina, e quando a IA alucina informações que parecem verdadeiras mas não são.
5. **O que significa utilizar uma LLM como apoio e não como substituta do raciocínio?** Significa nunca deixar de pensar criticamente à cerca de cada trabalho que fazemos com a IA e sobre as informações que ela dá, pois como foi dito no tópico anterior, ela pode alucinar e dar uma resposta completamente errada sobre um assunto, exalando uma enorme certeza. Cabe a nós programadores/cientistas avaliarmos criticamente cada prompt que a IA nos oferece, seja em questão de eficiência, de memória, de veracidade de fatos, etc.